



وزارة التعليم العالي و البحث العلمي

جامعة دمشق

كلية الهندسة المعلوماتية - قسم الذكاء الصناعي

مشروع التجارة الالكترونية - البرمجة التفرعية

وظيفة مشروع البرمجة التفرعية

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

إعداد الطلاب:

إسراء علي عكاشه

أحمد سليمان اسماعيل

مرام ياسين ياسين

مجد إياد خالد

ابراهيم عبد الرحمن احمد(شبكات)

يارا مرسل الحمود هنيدي

مقدمة

يهدف المشروع إلى بناء **واجهة برمجية خلفية (RESTful API)** لنظام تجارة إلكترونية قادر على التعامل مع آلاف الطلبات المتزامنة بكفاءة واستقرار. المشروع مبني بإطار عمل **Laravel 13** مع قاعدة بيانات **MySQL**، ويركز على تطبيق المتطلبات غير الوظيفية المرتبطة بالبرمجة التفرعية أكثر من التركيز على كثرة الوظائف.

الهدف الأساسي هو إثبات قدرة النظام على الصمود تحت الضغط العالي، من خلال ثلاثة محاور رئيسية يتم شرحها تفصيلياً في هذا الوثيقة.

التقنيات المستخدمة

التقنية	الاصدار	الغرض
LARAVEL 13	13.8	اطار العمل الرئيسي
MySql	-	قاعدة البيانات المستخدمة
Laravel Sanctum	4.3	مصادقة API بواسطة Personal Access Tokens
Database Queue Driver	مدمج مع Laravel	المعالجة غير المتزامنة للمهام الخلفية (Jobs)
Laravel Cache (Database)	مدمج مع Laravel	تخزين عداد طلبات Semaphore لـ ConcurrencyLimiter

الميزات الوظيفية

المصادقة : انشاء حساب , تسجيل دخول , معلومات المستخدم الحالي , تسجيل خروج

الميزة	النوع	المسار
انشاء حساب	post	/api/auth/register
تسجيل الدخول	post	/api/auth/login
معلوماتي	get	/api/auth/me
خروج	post	/api/auth/logout

المنتجات :

المسار	النوع	الميزة
/api/products	get	قائمة المنتجات (Paginated, فلترة بـ ?category أو ?search)
/api/products/{id}	get	تفاصيل منتج واحد
/api/products	post	انشاء منتج
/api/products/{id}	put	تحديث بيانات المنتج
/api/products/{id}	delete	إلغاء تفعيل منتج (soft delete – is_active=false)

سلة التسوق

المسار	النوع	الميزة
/api/cart	get	عرض السلة مع العناصر و الاجمالي
/api/cart/items	post	اضافة منتج للسلة
/api/cart/items/{id}	put	تعديل كمية عنصر (0 = حذف)
/api/cart/items/{id}	delete	حذف عنصر من السلة
/api/cart	delete	افراغ سلة بالكامل

الطلبات

المسار	النوع	الميزة
/api/orders	get	قائمة طلبات المستخدم
/api/orders	post	اتمام الطلب من السلة
/api/orders/{id}	get	تفاصيل طلب واحد مع الفاتورة
/api/orders/{id}/cancel	patch	الغاء طلب بحالة pending

ادارة المخزون

الميزة	النوع	المسار
المنتجات ذات المخزون ≥ 10 وحدات	get	/api/inventory/low-stock
إضافة كمية لمخزن منتج (مع lockForUpdate)	post	/api/inventory/{id}/restock
ضبط المخزون لكمية محددة (مع lockForUpdate)	put	/api/inventory/{id}/adjust

و يوجد Apis سيتم انشاؤها لتجريب الميزات الغير وظيفية , عموما كل توثيق ال api سنرفقه في كوليكتشن لبوستان داخل المشروع

الميزات غير الوظيفية

1 - حماية البيانات من التضارب (Race Condition)

المشكلة : بفرض دخل ثلاث مستخدمين هم A B C و كلهم يريدون المنتج P الذي لا يوجد غيره في المخزن السيناريو :

A قرأ P حيث $stock = 1$ (شرط وجود منتج محقق)
B قرأ p حيث $stock = 1$ (شرط وجود منتج محقق)
C قرأ P حيث $stock = 1$ (شرط وجود منتج محقق)

A طلب P اذا $stock = 1 - 1$
B طلب P اذا $stock = 0 - 1$
C طلب P اذا $stock = -1 - 1$

و stock أصبح سالب 2 و تم بيع منتجات غير موجودين

الحل :

في ملف app/Services/OrderService.php في السطر 44

```
$product = Product::where('id', $cartItem->product_id)
```

```
->lockForUpdate()
```

```
->firstOrFail();
```

تم تطبيق دالة lockForUpdate على استدعاء المنتج و التي توازي امر ال sql التالي

```
SELECT * FROM products WHERE id=? FOR UPDATE
```

بالتالي

1- المستخدم A يبدأ Transaction ويُنفذ SELECT ... FOR UPDATE فيحجز القفل على صف المنتج

المستخدم B يحاول SELECT ... FOR UPDATE على نفس الصف → يتوقف و ينتظر (Blocked)

المستخدم C نفس الشيء → ينتظر (Blocked)

2- المستخدم A يُنقص المخزون و يُنهي ال Transaction ثم يُحرر القفل

المستخدم B يستيقظ و يقرأ المخزون المحدث = 0 و يفشل ب Insufficient Stock

المستخدم B يستيقظ و يقرأ المخزون المحدث = 0 و يفشل ب Insufficient Stock

النتيجة مع locking: المخزون = 0 بالضبط. مستخدم واحد فقط نجح في الشراء. لا oversell.

تم اختبار هذا السيناريو فعلياً: 3 مستخدمين يطلبون نفس المنتج (stock=1) في نفس اللحظة بالتالي 1 نجح فقط، 2 رفضوا ب HTTP 409.

2 - ادارة الموارد و ضبط الطاقة الاستيعابية

إذا وصل 1000 طلب لـ Checkout في نفس اللحظة بدون أي قيود، سيفتح النظام 1000 اتصال بقاعدة البيانات في نفس اللحظة، مما يستنفد ال Connection Pool و يُعطل الخادم بالكامل.

الحل 1 : محدد المعدل (60 طلب/دقيقة)

و هو موجود افتراضياً في Laravel حيث يوفر throttle middleware مدمجاً يحدد عدد الطلبات لكل مستخدم: في ملف bootstrap/app.php

```
->withMiddleware(function (Middleware $middleware): void {
```

```
    $middleware->statefulApi(); // تلقائياً throttle:api يُفعل
```

}}

الحل 2 : محدد التزامن (نمط الإشارة)

ملف مخصص: app/Http/Middleware/ConcurrencyLimiter.php – يعمل كـ Counting Semaphore بمستوى التطبيق

مثال للاستخدام

file : routes/api.php

```
Route::post('/', [OrderController::class, 'store'])
```

```
->middleware(ConcurrencyLimiter::class . ':5') // max 5 طلبات متزامنة
```

```
->name('orders.store');
```

maxConcurrent = 5 على endpoint Checkout: أكثر عملية حرجة تستهلك DB connections

ال finally block يضمن تحرير العداد حتى لو رمى الطلب exception – لا resource leak
النمط مشابه لـ Counting Semaphore في نظم التشغيل: يسمح بـ N thread في الوقت ذاته

3 - المعالجة غير المتزامنة (Async Queues)

المشكلة – المهام الثقيلة تُبطئ الاستجابة بعد إتمام الطلب، النظام يحتاج إلى: إنشاء فاتورة PDF + إرسال بريد إلكتروني + فحص المخزون. إذا نُفذت هذه العمليات داخل الـ HTTP request، سيتأخر الرد للمستخدم عدة ثوانٍ.

الحل – Laravel Queue System

الملف: `app/Jobs/GenerateInvoiceJob.php`

- المهمة: ينشئ سجل invoice في قاعدة البيانات برقم فاتورة فريد (INV-XXXXXXXX-ID)
- المحاولات: 3 = tries إذا فشل يُعاد تشغيله 3 مرات تلقائياً
- الحماية: يتحقق أن الفاتورة لم تُنشأ مسبقاً (idempotent) لتجنب التكرار

SendOrderNotificationJob – إرسال الإشعارات

الملف: `app/Jobs/SendOrderNotificationJob.php`

- المهمة: إرسال بريد تأكيد الطلب للعميل
- المحاولات: 5 = tries, 60 = backoff ثانية بين كل محاولة
- في الإنتاج: يُستبدل `Log::info` بـ `Mail::to()->send`

NotifyLowStockJob – تنبيه نقص المخزون

الملف: `app/Jobs/NotifyLowStockJob.php`

- المهمة: إذا أصبح المخزون ≥ 5 وحدات، يُسجل تحذير للمدير
- الحد: 5 = LOW_STOCK_THRESHOLD وحدات

4 – معالجة البيانات الضخمة على دفعات (Batch Processing)

المشكلة – تقارير المبيعات تحوي على بيانات ضخمة

تشغيل `Order::get()` على 100,000 سجل يُحمّل جميعها في الذاكرة دفعةً واحدة: طريقة خاطئة – تُحمّل كل البيانات في RAM :

```
$orders = Order::whereDate('created_at', $date)->get(); // 500 MB+ قد يستهلك
```

الحل – 100 Chunks سجل في الذاكرة في أي لحظة
كل chunk يُعالج بشكل مستقل بواسطة job منفصل

المعمارية – نمط Orchestrator + Parallel Chunks

3 jobs تعمل بالتسلسل والتوازي معاً:

1. يجلب IDs فقط من قاعدة البيانات (بدون تحميل البيانات)
2. يقسم IDs إلى chunks بحجم 100
3. يُطلق كل chunk كـ job مستقل عبر `Bus::batch()`

5 - توزيع الأحمال (Load Distribution)

في بيئة الإنتاج يكون Load Balancer خارجياً (Nginx / HAProxy / AWS ALB). هنا نطبق المنطق الداخلي لكل استراتيجية ونُمثل الخوادم بـ Queue channels مستقلة، كل channel يُعالج بواسطة worker على خادم مختلف.

الخوادم المُعرَّفة (Nodes)

الخوادم nodes	السعة	الوزن	Queue Channel	نسبة الحركة (Weighted)
node-1	100 طلب	3	worker-1	50%
node-2	80 طلب	2	worker-2	33.3%
node-3	50 طلب	1	worker-3	16.7%

الاستراتيجيات الأربع المُطبَّقة

الاستراتيجية	المبدأ	نتيجة 6 طلبات	الافضل ل
Round Robin	كل طلب يذهب للخادم التالي في الدورة	2 + 2 + 2 (33% لكل)	طلبات متماثلة
Weighted Round Robin	الأقوى يستقبل نسبة أعلى حسب الوزن	1 + 2 + 3 (50/33/17%)	خوادم غير متساوية
Least Connections	الطلب لأقل خادم انشغالاً	يعتمد على الحمل الحالي	طلبات بأوزان مختلفة
Random	اختيار عشوائي	غير محدد	بسيط / تجريبي

